Assignment 6

DATA 260

Name: Kazi Samina Maraj Mumu

SJSU ID: 018324098

Date: 10/16/2026

GitHub repository: https://github.com/SaminaMaraj/data260-4098

Personal Configuration and Project Information

Item	Value
Domain	Municipal transit incidents
Repository	github.com/SaminaMaraj/data260-4098
Final commit	d756ec5362f0050acc0895719109afb9d57261ff
Tag	hw6
Local model	qwen3:8b via Ollama
Hardware	Apple MacBook Pro, M5 chip, 24 GB RAM
Python	3.12.10
Database	MongoDB 7 (Docker), database s4098_doc

SID4	PORT_BASE	PREFIX	SEED	VERIFY_SEED	DOMAIN_ID
4098	8498	s4098	4098	264098	2

Submission Details

Repository: https://github.com/SaminaMaraj/data260-4098. Final commit: d756ec5362f0050acc0895719109afb9d57261ff, tagged hw6. The repository link was checked and opens the public repository.

This homework extends the transit incident application with a MongoDB document API and a persistent memory assistant. I tested CRUD operations, Pydantic validation, memory retrieval, summaries, embeddings, file-based memory commands, and memory-size experiments.

Part 1 - MongoDB Document API

I created the s4098_doc MongoDB database with an incidents collection. The incident model requires an incident title (maximum 100 characters), route line, category, and occurrence time. The description is optional. Category is restricted to delay, collision, service-suspension, and infrastructure-issue. Status defaults to open and can be open, investigating, or resolved. created_at and updated_at are set by the server.

Implementation

src/mongo_config.py and src/mongo_schemas.py - configuration and Pydantic schema

```
MONGO_URI = os.environ.get("MONGO_URI", "mongodb://localhost:27017")
MONGO_DB_NAME = os.environ.get("MONGO_DB_NAME", "s4098_doc")
MONGO_INCIDENTS_COLLECTION = "incidents"

IncidentStatus = Literal["open", "investigating", "resolved"]
IncidentCategory = Literal[
    "delay",
    "collision",
    "service-suspension",
    "infrastructure-issue",
]

class MongoIncidentCreate(BaseModel):
    incident_title: str = Field(..., min_length=1, max_length=100)
    route_line: str = Field(..., min_length=1, max_length=100)
    description: str | None = Field(default=None, max_length=2000)
    status: IncidentStatus = "open"
    category: IncidentCategory
    occurred_at: datetime
```

src/mongo_db.py - PyMongo connection and MongoDB-side validation (excerpt)

```
def get_mongo_client() -> MongoClient:
    """Return one reusable client without connecting during app import."""
    global _client
    if _client is None:
        _client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=3000)
    return _client

def _incident_validator() -> dict:
    """MongoDB-side validation that mirrors the Pydantic API schema."""
    return {
        "$jsonSchema": {
            "bsonType": "object",
            "required": ["incident_title", "route_line", "status", "category",
                         "occurred_at", "created_at", "updated_at"],
            "properties": {
                "incident_title": {"bsonType": "string", "maxLength": 100},
                "description": {"bsonType": ["string", "null"], "maxLength": 2000},
                "status": {"enum": ["open", "investigating", "resolved"]},
                # ... category enum, route_line, and date fields omitted
                "created_at": {"bsonType": "date"},
                "updated_at": {"bsonType": "date"},
            },
        }
    }

# inside get_incidents_collection():
db.create_collection(
    MONGO_INCIDENTS_COLLECTION,
    validator=_incident_validator(),
```

```
    validationLevel="strict",
    validationAction="error",
)
```

src/api.py - CORS middleware (FastAPI parses JSON bodies into the Pydantic models)

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

src/mongo_api.py - ObjectId handling, timestamps, CRUD handlers, and HTTP errors

```
def _object_id(value: str) -> ObjectId:
    if not ObjectId.is_valid(value):
        raise HTTPException(status_code=400, detail="Invalid MongoDB id format.")
    return ObjectId(value)

def _not_found() -> HTTPException:
    return HTTPException(status_code=404, detail="MongoDB incident not found.")

@router.post("", response_model=MongoIncidentOut, status_code=status.HTTP_201_CREATED)
def create_mongo_incident(payload: MongoIncidentCreate) -> dict:
    document = payload.model_dump()
    timestamp = _now()
    document["created_at"] = timestamp
    document["updated_at"] = timestamp
    try:
        collection = get_incidents_collection()
        result = collection.insert_one(document)
        created = collection.find_one({"_id": result.inserted_id})
        if created is None:
            raise HTTPException(status_code=500,
                                detail="Created document could not be read.")
        return _serialize(created)
    except HTTPException:
        raise
    except PyMongoError as exc:
        if getattr(exc, "code", None) == 11000:
            raise HTTPException(
                status_code=409,
                detail="An incident with the same unique value already exists.",
            ) from exc
        raise HTTPException(status_code=503,
                            detail=f"MongoDB error while creating incident: {exc}") from exc

@router.get("", response_model=list[MongoIncidentOut])
def list_mongo_incidents() -> list[dict]:
    try:
        documents = get_incidents_collection().find().sort("created_at", -1)
        return [_serialize(document) for document in documents]
    except PyMongoError as exc:
        raise HTTPException(status_code=503,
                            detail=f"MongoDB error while listing incidents: {exc}") from exc

@router.get("/{incident_id}", response_model=MongoIncidentOut)
def get_mongo_incident(incident_id: str) -> dict:
    object_id = _object_id(incident_id)
    try:
        document = get_incidents_collection().find_one({"_id": object_id})
        if document is None:
            raise _not_found()
```

```
        return _serialize(document)
    except HTTPException:
        raise
    except PyMongoError as exc:
        raise HTTPException(status_code=503,
                            detail=f"MongoDB error while reading incident: {exc}") from exc

@router.put("/{incident_id}", response_model=MongoIncidentOut)
def update_mongo_incident(incident_id: str, payload: MongoIncidentUpdate) -> dict:
    object_id = _object_id(incident_id)
    updates = payload.model_dump(exclude_unset=True)
    if not updates:
        raise HTTPException(status_code=400,
                            detail="At least one field is required for an update.")
    updates["updated_at"] = _now()
    try:
        collection = get_incidents_collection()
        if collection.find_one({"_id": object_id}) is None:
            raise _not_found()
        collection.update_one({"_id": object_id}, {"$set": updates})
        updated = collection.find_one({"_id": object_id})
        if updated is None:
            raise _not_found()
        return _serialize(updated)
    except HTTPException:
        raise
    except PyMongoError as exc:
        raise HTTPException(status_code=503,
                            detail=f"MongoDB error while updating incident: {exc}") from exc

@router.delete("/{incident_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_mongo_incident(incident_id: str) -> None:
    object_id = _object_id(incident_id)
    try:
        result = get_incidents_collection().delete_one({"_id": object_id})
        if result.deleted_count == 0:
            raise _not_found()
    except HTTPException:
        raise
    except PyMongoError as exc:
        raise HTTPException(status_code=503,
                            detail=f"MongoDB error while deleting incident: {exc}") from exc
```

CRUD and validation results

- GET all incidents and GET one incident by MongoDB ObjectId returned the stored document.
- POST created an incident and returned HTTP 201 Created.
- PUT updated the incident status to resolved and returned HTTP 200 OK.
- DELETE returned HTTP 204 No Content.
- A missing required field, an invalid category value, and a number in place of the title string each returned HTTP 422 with a clear Pydantic error message.

```
(base) samina@Kazis-MacBook-Pro-2 ~ % curl -s http://127.0.0.1:8498/api/mongo/incidents | python -m json.tool
[
    {
        "id": "6acfdc4fd13721eb85bb4317",
        "incident_title": "Signal problem near Diridon Station",
        "route_line": "VTA Green Line",
        "description": "A signal problem caused a temporary service interruption.",
        "status": "investigating",
        "category": "infrastructure-issue",
        "occurred_at": "2026-10-14T11:30:00",
        "created_at": "2026-10-14T19:47:27.272000",
        "updated_at": "2026-10-14T19:47:27.272000"
    }
]
(base) samina@Kazis-MacBook-Pro-2 ~ % curl -s http://127.0.0.1:8498/api/mongo/incidents/6acfdc4fd13721eb85bb4317 | python -m json.tool
{
    "id": "6acfdc4fd13721eb85bb4317",
    "incident_title": "Signal problem near Diridon Station",
    "route_line": "VTA Green Line",
    "description": "A signal problem caused a temporary service interruption.",
    "status": "investigating",
    "category": "infrastructure-issue",
    "occurred_at": "2026-10-14T11:30:00",
    "created_at": "2026-10-14T19:47:27.272000",
    "updated_at": "2026-10-14T19:47:27.272000"
}
(base) samina@Kazis-MacBook-Pro-2 ~ % curl -s http://127.0.0.1:8498/api/mongo/incidents/6acfdc4fd13721eb85bb4317 | python -m json.tool
{
    "id": "6acfdc4fd13721eb85bb4317",
    "incident_title": "Signal problem near Diridon Station",
    "route_line": "VTA Green Line",
    "description": "A signal problem caused a temporary service interruption.",
    "status": "investigating",
    "category": "infrastructure-issue",
    "occurred_at": "2026-10-14T11:30:00",
    "created_at": "2026-10-14T19:47:27.272000",
    "updated_at": "2026-10-14T19:47:27.272000"
}
(base) samina@Kazis-MacBook-Pro-2 ~ %
```

Figure 1. GET all incidents and GET one incident by ObjectId.



Figure 2. POST request that created an incident (HTTP 201 Created).

```
(base) samina@Kazis-MacBook-Pro-2 ~ % curl -i -X PUT http://127.0.0.1:8498/api/mongo/incidents/6acfde7ad13721eb85bb4318 \
-H "Content-Type: application/json" \
-d '{
  "status": "resolved",
  "description": "The delay was resolved successfully."
}'
HTTP/1.1 200 OK
date: Wed, 14 Oct 2026 19:58:05 GMT
server: uvicorn
content-length: 325
content-type: application/json

{"id":"6acfde7ad13721eb85bb4318","incident_title":"Bus delay at Santa Clara Station","route_line":"VTA Route 22","description":"The delay was resolved successfully.","status":"resolved","category":"delay","occurred_at":"2026-10-14T10:00:00","created_at":"2026-10-14T19:56:42.585000","updated_at":"2026-10-14T19:58:05.967000"}%
(base) samina@Kazis-MacBook-Pro-2 ~ %
```

Figure 3. PUT request that updated the incident (HTTP 200 OK).

```
{"id":"6acfde7ad13721eb85bb4318","incident_title":"Bus delay at Santa Clara Station","route_line":"VTA Route 22","description":"The delay was resolved successfully.","status":"resolved","category":"delay","occurred_at":"2026-10-14T10:00:00","created_at":"2026-10-14T19:56:42.585000","updated_at":"2026-10-14T19:58:05.967000"}%
(base) samina@Kazis-MacBook-Pro-2 ~ % curl -i -X DELETE http://127.0.0.1:8498/api/mongo/incidents/6acfde7ad13721eb85bb4318
HTTP/1.1 204 No Content
date: Wed, 14 Oct 2026 19:59:57 GMT
server: uvicorn
content-type: application/json

(base) samina@Kazis-MacBook-Pro-2 ~ %
```

Figure 4. DELETE request (HTTP 204 No Content).

```
(base) samina@Kazis-MacBook-Pro-2 ~ % curl -i -X DELETE http://127.0.0.1:8498/api/mongo/incidents/6acfde7ad13721eb85bb4318
HTTP/1.1 204 No Content
date: Wed, 14 Oct 2026 19:59:57 GMT
server: uvicorn
content-type: application/json

(base) samina@Kazis-MacBook-Pro-2 ~ % curl -i -X POST http://127.0.0.1:8498/api/mongo/incidents \
-H "Content-Type: application/json" \
-d '{
  "route_line": "VTA Route 22",
  "category": "delay",
  "occurred_at": "2026-10-14T10:00:00Z"
}'
HTTP/1.1 422 Unprocessable Entity
date: Wed, 14 Oct 2026 20:01:35 GMT
server: uvicorn
content-length: 180
content-type: application/json

{"detail":[{"type":"missing","loc":["body","incident_title"],"msg":"Field required","input":{"route_line":"VTA Route 22","category":"delay","occurred_at":"2026-10-14T10:00:00Z"}}]}%
(base) samina@Kazis-MacBook-Pro-2 ~ %
```

Figure 5. Validation error for a request missing incident_title (HTTP 422).

```
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % docker exec -it data260-4098-mongodb mongosh s4098_doc --quiet --eval 'printjson(db.getCollectionNames()); printjson(db.incidents.findOne())'
[
  'incidents',
  'summaries',
  'messages',
  'episodes'
]
{
  _id: ObjectId('6acfdc4fd13721eb85bb4317'),
  incident_title: 'Signal problem near Diridon Station',
  route_line: 'VTA Green Line',
  description: 'A signal problem caused a temporary service interruption.',
  status: 'investigating',
  category: 'infrastructure-issue',
  occurred_at: ISODate('2026-10-14T11:30:00.000Z'),
  created_at: ISODate('2026-10-14T19:47:27.272Z'),
  updated_at: ISODate('2026-10-14T19:47:27.272Z')
}

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug data260-4098-mongodb
    Learn more at https://docs.docker.com/go/debug-cli/
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %
```

Figure 6. mongosh inspection of s4098_doc: collections and a sample incident with ObjectId and ISODate timestamps.



Figure 7. Invalid category value and incorrect title type, both rejected with HTTP 422.

Part 2 - Persistent MongoDB Memory

The chat endpoint stores user and assistant messages in the messages collection. Memory is scoped by user_id and session_id. Each prompt is assembled from the system primer, the lifetime and session summaries, the short-term window of the last SHORT_TERM_N messages, and the top-k episodic facts. Episodic facts are embedded with a deterministic hashed bag-of-words vector (32 dimensions, SHA-256 token hashing), not a hosted embedding model, and are retrieved by cosine similarity.

A session summary is created every SUMMARIZE_EVERY_USER_MSGS (5) user messages. Every second time this happens, recent session summaries are condensed into a lifetime summary with scope user and session_id null. In the test session the memory endpoint later returned the last 16 messages, 11 episodic facts, a session summary, and a lifetime summary.

Implementation

src/hw6_memory_api.py - POST /api/chat

```
@router.post("/chat")
def chat(payload: ChatRequest) -> dict:
    started = time.perf_counter()
    try:
        record_message(payload.user_id, payload.session_id, "user", payload.message)
        prompt_messages, accounting = build_prompt(
            payload.user_id, payload.session_id, payload.message
        )
        model_result = complete(prompt_messages)
        reply = model_result.get("content", "").strip()
        if not reply:
            raise HTTPException(status_code=502,
                                detail="The local model returned an empty reply.")
        record_message(payload.user_id, payload.session_id, "assistant", reply)
        save_episode_facts(payload.user_id, payload.session_id, payload.message)
        summaries_created = maybe_create_summaries(payload.user_id, payload.session_id)
        accounting.update(
            {
                "prompt_tokens": model_result.get("input_tokens", 0),
                "completion_tokens": model_result.get("output_tokens", 0),
                "total_tokens": model_result.get("total_tokens", 0),
                "latency_ms": round((time.perf_counter() - started) * 1000, 2),
```

```
                "model": model_result.get("model"),
            }
        )
        return {
            "reply": reply,
            "memory_used": accounting,
            "summaries_created": len(summaries_created),
        }
    except HTTPException:
        raise
    except PyMongoError as exc:
        raise HTTPException(status_code=503,
                            detail=f"MongoDB error in chat: {exc}") from exc
    except Exception as exc:
        raise HTTPException(status_code=503,
                            detail=f"Chat model error: {exc}") from exc
```

src/hw6_memory_api.py - GET /api/memory/{user_id} and GET /api/aggregate/{user_id}

```
@router.get("/memory/{user_id}")
def get_memory(user_id: str) -> dict:
    try:
        messages = list(
            get_messages_collection()
            .find({"user_id": user_id})
            .sort("created_at", -1)
            .limit(16)
        )
        messages.reverse()
        session_summary = get_summaries_collection().find_one(
            {"user_id": user_id, "scope": "session"}, sort=[("created_at", -1)]
        )
        lifetime_summary = get_summaries_collection().find_one(
            {"user_id": user_id, "scope": "user", "session_id": None},
            sort=[("created_at", -1)],
        )
        episodes = list(
            get_episodes_collection()
            .find({"user_id": user_id})
            .sort("created_at", -1)
            .limit(20)
        )
        return {
            "messages": [_iso(item) for item in messages],
            "latest_session_summary": _iso(session_summary),
            "latest_lifetime_summary": _iso(lifetime_summary),
            "episodes": [_iso(item) for item in episodes],
        }
    except PyMongoError as exc:
        raise HTTPException(status_code=503,
                            detail=f"MongoDB error while reading memory: {exc}") from exc

@router.get("/aggregate/{user_id}")
def get_aggregate(user_id: str) -> dict:
    try:
        daily_counts = list(
            get_messages_collection().aggregate(
                [
                    {"$match": {"user_id": user_id}},
                    {
                        "$group": {
                            "_id": {
                                "$dateToString": {"format": "%Y-%m-%d",
                                                  "date": "$created_at"}
                            },
```

```
                            "count": {"$sum": 1},
                        }
                    },
                    {"$sort": {"_id": 1}},
                ]
            )
        )
        summaries = list(
            get_summaries_collection()
            .find({"user_id": user_id})
            .sort("created_at", -1)
            .limit(5)
        )
        return {
            "daily_message_counts": [
                {"date": item["_id"], "count": item["count"]} for item in daily_counts
            ],
            "recent_summaries": [_iso(item) for item in summaries],
        }
    except PyMongoError as exc:
        raise HTTPException(status_code=503,
                            detail=f"MongoDB error while aggregating memory: {exc}") from exc
```

src/hw6_memory.py - episode extraction, embedding, and cosine-similarity retrieval

```
def extract_episode_facts(message: str) -> list[tuple[str, float]]:
    """Extract up to three short, user-originated facts with simple scores."""
    candidates = [
        re.sub(r"\s+", " ", part).strip(" .!?\n")
        for part in re.split(r"[.!?\n]+", message)
    ]
    facts: list[tuple[str, float]] = []
    for candidate in candidates:
        words = candidate.split()
        if len(words) < 3:
            continue
        # ... facts longer than 240 characters are truncated here
        importance = 0.8 if re.search(
            r"\b(i am|i'm|my goal|i need|i prefer|i work|remember)\b",
            candidate,
            re.IGNORECASE,
        ) else 0.5
        facts.append((candidate, importance))
        if len(facts) == 3:
            break
    return facts

def _hash_embedding(text: str) -> list[float]:
    vector = [0.0] * EMBEDDING_DIMENSIONS          # EMBEDDING_DIMENSIONS = 32
    tokens = re.findall(r"[a-z0-9]+", text.casefold())
    for token in tokens:
        digest = hashlib.sha256(token.encode("utf-8")).digest()
        index = int.from_bytes(digest[:4], "big") % EMBEDDING_DIMENSIONS
        vector[index] += 1.0
    magnitude = math.sqrt(sum(value * value for value in vector))
    if magnitude:
        vector = [value / magnitude for value in vector]
    return vector

def cosine_similarity(left: list[float], right: list[float]) -> float:
    if not left or not right:
        return 0.0
    size = min(len(left), len(right))
    return sum(left[i] * right[i] for i in range(size))
```

```
def retrieve_episodes(user_id: str, message: str,
                      top_k: int = EPISODIC_TOP_K) -> list[dict]:
    query_vector = _hash_embedding(message)
    scored: list[tuple[float, dict]] = []
    for document in get_episodes_collection().find({"user_id": user_id}):
        score = cosine_similarity(query_vector, document.get("embedding", []))
        scored.append((score, document))
    scored.sort(key=lambda item: (item[0], item[1].get("created_at", utc_now())),
                reverse=True)
    results = []
    for score, document in scored[:top_k]:
        item = dict(document)
        item["similarity"] = round(score, 4)
        results.append(item)
    return results
```

src/hw6_memory.py - session-summary trigger and lifetime-summary creation

```
def maybe_create_summaries(user_id: str, session_id: str | None) -> list[dict]:
    if session_id is None:
        return []
    messages_collection = get_messages_collection()
    user_count = messages_collection.count_documents(
        {"user_id": user_id, "session_id": session_id, "role": "user"}
    )
    if user_count == 0 or user_count % SUMMARIZE_EVERY_USER_MSGS != 0:
        return []
    # ... load recent messages and the latest session summary
    text, _ = _generate_summary(recent_messages, (latest or {}).get("text", ""))
    summary = {"user_id": user_id, "session_id": session_id, "scope": "session",
               "text": text, "created_at": utc_now()}
    get_summaries_collection().insert_one(summary)

    if user_count % (SUMMARIZE_EVERY_USER_MSGS * 2) == 0:
        prior = list(
            get_summaries_collection()
            .find({"user_id": user_id, "scope": "session"})
            .sort("created_at", -1)
            .limit(10)
        )
        lifetime_text, _ = _generate_summary(
            [{"role": "summary", "content": item.get("text", "")} for item in prior]
        )
        lifetime = {"user_id": user_id, "session_id": None, "scope": "user",
                    "text": lifetime_text, "created_at": utc_now()}
        get_summaries_collection().insert_one(lifetime)
        return [summary, lifetime]
    return [summary]
```

src/hw6_memory.py - prompt assembly (build_prompt, excerpt)

```
window = load_short_term_window(user_id, session_id)
summaries = load_latest_summaries(user_id, session_id)
episodes = retrieve_episodes(user_id, current_message)
# ... lifetime_text, session_text, episodic_text, window_text built from the above
long_term_text = f"Lifetime:\n{lifetime_text or 'None'}\nSession:\n{session_text or 'None'}"
context_text = (
    f"Long-term summaries:\n{long_term_text}\n\n"
    f"Short-term window ({len(window)} messages):\n{window_text}\n\n"
    f"Episodic facts:\n{episodic_text}\n\n"
    f"Current user message:\n{current_message}"
)
prompt_messages = [
    {"role": "system", "content": SYSTEM_PRIMER},
    {"role": "user", "content": context_text},
```

```
]
accounting = {
    "system_primer_tokens": _estimate_tokens(SYSTEM_PRIMER),
    "long_term_summary_tokens": _estimate_tokens(long_term_text),
    "short_term_window_tokens": _estimate_tokens(window_text),
    "episodic_fact_tokens": _estimate_tokens(episodic_text),
    # ... plus the message list, summaries, and facts returned to the client
}
```

Results



Figure 8. Successful POST /api/chat: command, reply, component token counts, prompt_tokens, latency_ms, and model.



Figure 9. Later chat turn (start and end of one response): short-term window, three episodic facts retrieved by cosine similarity, and summaries_created = 1.

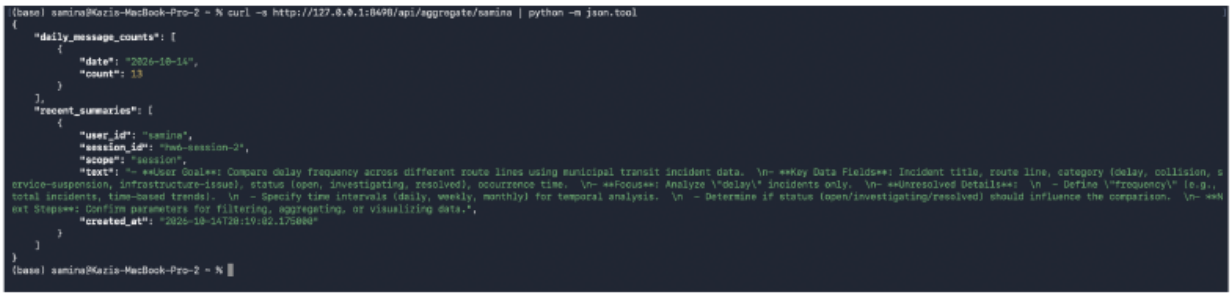


Figure 10. GET /api/aggregate/samina: daily message counts and the recent session summary.

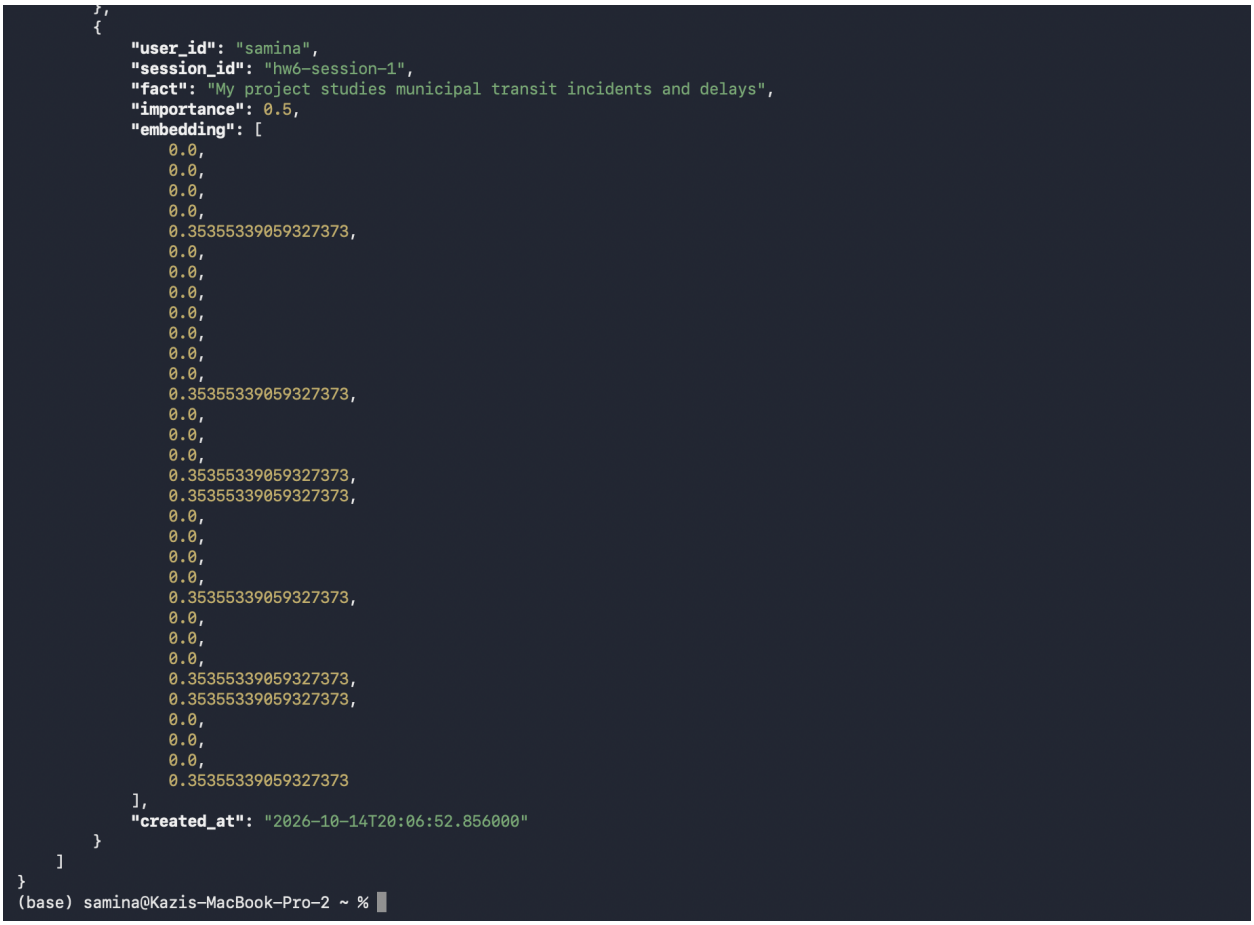


Figure 11. GET /api/memory/samina (end of output): a stored episodic fact with its importance score and embedding vector.



Figure 12. GET /api/memory/samina summarized: 16 messages, the latest session summary, the lifetime summary (scope user, session_id null), and 11 episodes.



Figure 13. mongosh samples from the messages, summaries, and episodes collections.

Part 3 - Memory Experiments and Cost

For every setting, I ran the same 30-message conversation in a fresh session, followed by the same five memory probes. The conversation is the fixed CONVERSATION list in code/hw6_experiments.py, and each setting uses a new user_id, so no memory carries over between settings. Every /api/chat call logs prompt and completion tokens, the tokens used by the system primer, long-term summary, short-term window, and episodic facts, and the request latency.

The raw results are stored in reports/hw06/raw/: memory_turns.csv and memory_turns.json (120 per-turn rows, 30 per setting), memory_probes.json (probe replies and scores), and memory_cost_summary.json (the summary table below). The console output of the runs is recorded in reports/hw06/RUN_LOG.txt.

Setting	Mean prompt to kens/turn	Mean completion tokens	Mean latency (ms)	Probe quality (1-5)	Cost for 10,000 conversations
SHORT_TERM_N=4	544.10	47.53	5211.51	3.0	$234.53
SHORT_TERM_N=12	761.20	39.33	5359.90	3.0	$287.36
SHORT_TERM_N=30	1087.83	41.80	6444.42	3.4	$389.05
eviction_keep_12	761.20	39.33	5643.06	3.0	$287.36

Cost estimate

Cost for 10,000 conversations = (mean prompt tokens x 30 turns x 10,000 x $1 per million) + (mean completion tokens x 30 turns x 10,000 x $5 per million). For example, SHORT_TERM_N=4 costs 163.23 + 71.30 = $234.53.

Eviction policy

eviction_keep_12 runs with SHORT_TERM_N=30 configured but sets EVICTION_KEEP_N=12. Older messages stay in MongoDB, but only the most recent 12 are placed in the prompt. Because the cap is 12 messages, its prompt size and probe quality match SHORT_TERM_N=12 exactly. Compared with N=30, it saves 326.63 prompt tokens per turn and lowers the estimated cost from $389.05 to $287.36, but probe quality drops from 3.4 to 3.0.

src/hw6_memory.py - eviction cap applied to the short-term window

```
EVICTION_KEEP_N: int | None = None   # set to 12 for the eviction_keep_12 run

def load_short_term_window(user_id: str, session_id: str | None) -> list[dict]:
    # ... build the user/session query
    window_size = EVICTION_KEEP_N or SHORT_TERM_N
    documents = list(
        get_messages_collection()
        .find(query)
        .sort("created_at", -1)
        .limit(window_size)
    )
    documents.reverse()
    return documents
```

Selecting the best and worst settings

The script ranks settings by probe quality first and uses prompt tokens to break ties. The best setting has the highest probe quality, with fewer prompt tokens winning a tie. The worst setting has the lowest probe quality, with more prompt tokens losing a tie.

- Best: SHORT_TERM_N=30 has the highest probe quality (3.4). It also has the most prompt tokens, the highest latency, and the highest cost ($389.05).
- Worst: N=4, N=12, and eviction_keep_12 all score 3.0. N=12 and eviction_keep_12 use the most prompt tokens of that group (761.20, tied), and the script reports SHORT_TERM_N=12 because it comes first in the sorted list. N=12 costs $52.83 more than N=4 and is slower, with no quality gain.

- SHORT_TERM_N=4 is the cheapest and fastest setting at the same 3.0 quality, so it is the best choice when cost matters more than the 0.4-point quality gain from N=30.

The five probes test route recall, the project goal, the difficult topic, the response preference, and an important project fact. Each has expected criteria and a 1-5 rubric.

```
source .venv/bin/activate
[OLLAMA_TIMEOUT=90 MODEL_MAX_TOKENS=1024 PYTHONPATH=. PYTHONUNBUFFERED=1 python -u code/hw6_experiments.py
{
  "settings": [
    {
      "setting": "SHORT_TERM_N=12",
      "mean_prompt_tokens": 761.2,
      "mean_completion_tokens": 39.33,
      "mean_latency_ms": 5359.9,
      "mean_probe_quality": 3.0,
      "estimated_cost_10000_conversations": 287.355,
      "prompt_token_savings_vs_n30": 326.63
    },
    {
      "setting": "SHORT_TERM_N=30",
      "mean_prompt_tokens": 1087.83,
      "mean_completion_tokens": 41.8,
      "mean_latency_ms": 6444.42,
      "mean_probe_quality": 3.4,
      "estimated_cost_10000_conversations": 389.049,
      "prompt_token_savings_vs_n30": 0.0
    },
    {
      "setting": "SHORT_TERM_N=4",
      "mean_prompt_tokens": 544.1,
      "mean_completion_tokens": 47.53,
      "mean_latency_ms": 5211.51,
      "mean_probe_quality": 3.0,
      "estimated_cost_10000_conversations": 234.525,
      "prompt_token_savings_vs_n30": 543.73
    },
    {
      "setting": "eviction_keep_12",
      "mean_prompt_tokens": 761.2,
      "mean_completion_tokens": 39.33,
      "mean_latency_ms": 5643.06,
      "mean_probe_quality": 3.0,
      "estimated_cost_10000_conversations": 287.355,
      "prompt_token_savings_vs_n30": 326.63
    }
  ],
  "best_setting": "SHORT_TERM_N=30",
  "worst_setting": "SHORT_TERM_N=12"
}
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %
```

Figure 14. Experiment output with mean tokens, latency, probe quality, cost, token savings compared with N=30, and the selected best and worst settings.

```
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % sed -n '1,200p' memory_probes.yaml
probes:
  - id: route
    question: "Which transit route or line did I mention?"
    expected_criteria: [route, line, "VTA Route 22"]
    rubric: "1=no answer; 3=partially correct; 5=correct route with context"
  - id: goal
    question: "What is my main goal for this assistant?"
    expected_criteria: [goal, objective, reliable]
    rubric: "1=no answer; 3=general goal; 5=specific remembered goal"
  - id: difficulty
    question: "What transit issue or topic did I say was difficult?"
    expected_criteria: [difficult, episodic, retrieval]
    rubric: "1=no answer; 3=related issue; 5=the exact remembered issue"
  - id: preference
    question: "What response preference did I mention?"
    expected_criteria: [prefer, concise, example]
    rubric: "1=no answer; 3=general preference; 5=exact preference"
  - id: fact
    question: "What important fact about my project should you remember?"
    expected_criteria: [project, municipal, transit, MongoDB]
    rubric: "1=no answer; 3=related fact; 5=the specific project fact"
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %
```

Figure 15. memory_probes.yaml: five probes with expected criteria and a 1-5 rubric.

Part 4 - File-Based Memory

The file-based memory implementation loads AGENT.md and MEMORY.md, adds both to the system prompt, and decides that consolidation is required when MEMORY.md has more than 40 non-empty, non-comment lines or when its recorded consolidation time is more than 24 hours old. Consolidation merges duplicate lines, records each contradiction explicitly instead of silently deleting it, and then audits the latest-value resolutions against the project configuration.

Implementation

src/file_memory.py - loading files, building the prompt, and consolidation checks

```
  CONSOLIDATION_PATTERN = re.compile(r"last_consolidated_at:\s*(.+)", re.IGNORECASE)

  def load_memory_files(agent_path: Path = AGENT_PATH,
                        memory_path: Path = MEMORY_PATH) -> dict[str, str]:
      return {
          "agent": agent_path.read_text(encoding="utf-8") if agent_path.exists() else "",
          "memory": memory_path.read_text(encoding="utf-8") if memory_path.exists() else "",
      }

  def build_memory_system_prompt(agent_text: str, memory_text: str) -> str:
      return (
          "Durable agent instructions:\n"
          f"{agent_text.strip() or 'None'}\n\n"
          "Durable user/project memory:\n"
          f"{memory_text.strip() or 'None'}"
      )

  def memory_line_count(text: str) -> int:
      return sum(1 for line in text.splitlines()
                 if line.strip() and not line.lstrip().startswith("<!--"))

  def read_last_consolidated_at(text: str) -> datetime | None:
      match = CONSOLIDATION_PATTERN.search(text)
      if not match:
          return None
      try:
          # The timestamp is stored inside an HTML comment, so drop the closing "-->".
          timestamp = match.group(1).split("-->", 1)[0].strip()
          return datetime.fromisoformat(timestamp.replace("Z", "+00:00"))
      except ValueError:
```

```
        return None

def consolidation_required(memory_text: str, now: datetime | None = None,
                           max_lines: int = 40, max_age_hours: int = 24) -> bool:
    if memory_line_count(memory_text) > max_lines:
        return True
    last = read_last_consolidated_at(memory_text)
    if last is None:
        return True
    current = now or datetime.now(timezone.utc)
    if last.tzinfo is None:
        last = last.replace(tzinfo=timezone.utc)
    return current - last > timedelta(hours=max_age_hours)
```

src/file_memory.py - deduplication, contradiction detection, and the audit (compact_memory, excerpt)

```
for line in original.splitlines():
    stripped = line.strip()
    if not stripped or stripped.startswith("<!--"):
        continue
    normalized = _normalized(stripped)
    if normalized in seen:                      # duplicate fact -> merged
        duplicates.append(stripped)
        continue
    seen.add(normalized)
    key = _contradiction_key(stripped)          # e.g. "preferred model"
    if key and key in latest_by_key:
        previous_index, previous = latest_by_key[key]
        if previous != stripped and ":" in previous and ":" in stripped:
            contradictions.append({"key": key, "old": previous, "new": stripped,
                                   "resolution": "retained latest"})
            kept[previous_index] = (f"- Conflict resolved for {key}: retained latest; "
                                    f"superseded: {previous}")
    if key:
        latest_by_key[key] = (len(kept), stripped)
    kept.append(stripped)
# ... write the consolidated file with a last_consolidated_at timestamp
incorrectly_resolved = _audit_incorrect_resolutions(contradictions)

def _audit_incorrect_resolutions(
    contradictions: list[dict[str, str]],
) -> list[dict[str, str]]:
    """Flag latest-value resolutions that conflict with the active project configuration."""
    expected = {
        "preferred model": "qwen3:8b",
        "project status": "active",
    }
    incorrect = []
    for conflict in contradictions:
        expected_value = expected.get(conflict["key"])
        if expected_value and expected_value.casefold() not in conflict["new"].casefold():
            incorrect.append({**conflict,
                              "reason": f"expected configured value: {expected_value}"})
    return incorrect
```

src/file_memory.py - /memory, /compact, /dream, and /help

```
HELP_TEXT = {
    "/memory": "Displays the current AGENT.md and MEMORY.md contents.",
    "/compact": "Consolidates MEMORY.md and reports line counts before and after, "
                "duplicates merged, and contradictions found.",
    "/dream": "Extracts useful high-confidence facts from recent messages and adds them "
              "to MEMORY.md.",
    "/help": "Lists and explains the available memory commands.",
```

```
}

def handle_memory_command(
    command: str,
    recent_messages: list[str] | None = None,
    agent_path: Path = AGENT_PATH,
    memory_path: Path = MEMORY_PATH,
    output_path: Path | None = None,
) -> dict:
    command = command.strip().split()[0] if command.strip() else ""
    files = load_memory_files(agent_path, memory_path)
    if command == "/memory":
        return {"command": command, "agent": files["agent"], "memory": files["memory"],
                "memory_line_count": memory_line_count(files["memory"])}
    if command == "/compact":
        result = compact_memory(memory_path, output_path)
        result["command"] = command
        return result
    if command == "/dream":
        # Keep only high-confidence, useful facts instead of copying every recent message.
        facts = []
        for message in recent_messages or []:
            for fact, importance in extract_episode_facts(message):
                if importance >= 0.8:
                    facts.append(fact)
        facts = facts[:3]
        with memory_path.open("a", encoding="utf-8") as handle:
            for fact in facts:
                handle.write(f"\n- Recent fact: {fact}")
        return {"command": command, "facts_added": facts}
    if command == "/help":
        return {"command": command, "commands": HELP_TEXT}
    return {"command": command, "error": "Unknown memory command."}
```

Timestamp and /help tests

The consolidation timestamp is stored inside an HTML comment (<!-- last_consolidated_at: ... -->). The first version of read_last_consolidated_at also captured the closing -->, so parsing failed and consolidation_required() always returned True, even right after a consolidation. I fixed this by removing --> before parsing and added tests/test_file_memory_hw06.py. The tests check that a recent MEMORY.md with fewer than 40 non-comment lines returns False, and that an old timestamp or more than 40 lines returns True. They also check that /help explains every command.

Output of python tests/test_file_memory_hw06.py (two long lines are wrapped to fit the page)

```
Kazi Samina Maraj Mumu
PASS test_timestamp_inside_comment_is_parsed
PASS test_recent_and_short_memory_does_not_need_consolidation
PASS test_old_timestamp_needs_consolidation
PASS test_more_than_40_lines_needs_consolidation
PASS test_help_explains_every_command
{
  "command": "/help",
  "commands": {
    "/memory": "Displays the current AGENT.md and MEMORY.md contents.",
    "/compact": "Consolidates MEMORY.md and reports line counts before and after,
                 duplicates merged, and contradictions found.",
    "/dream": "Extracts useful high-confidence facts from recent messages and adds them
              to MEMORY.md.",
    "/help": "Lists and explains the available memory commands."
  }
```

```
}
```

Consolidation result

The original fixture has 55 lines, including three different realistic facts that are duplicated (19 duplicate lines in total) and three contradictory pairs. Consolidation reduced it to 37 counted lines, below the 40-line threshold. All three conflicts were recorded explicitly with the superseded value. The audit then found that two of the latest-value resolutions are incorrect.

Conflict	Old value	New value (retained)	Audit result
Response preference	concise	detailed	Correct: a later preference legitimately replaces the earlier one.
Preferred model	qwen3:8b	qwen3:4b	Incorrect: the project uses qwen3:8b.
Project status	active	paused	Incorrect: the project is active.

Final counts: before_lines 55, after_lines 37, duplicates_merged 19, contradictions 3, incorrectly_resolved 2. These two conflicts were not accepted as correct just because they appeared later in the file.

File evidence

The complete AGENT.md, MEMORY.original.md, and MEMORY.consolidated.md files are in the repository. Only the relevant excerpts are shown here.

AGENT.md (HW6 instructions; the four HW1 code-review lines are omitted)

```
- Project description: this repository implements a municipal transit incident API
  and memory assistant.
- Technology stack: keep the existing FastAPI, SQLAlchemy/MySQL, MongoDB, React, and
  Ollama components compatible.
- Coding convention: use clear snake_case Python names, typed function signatures,
  small modules, and helpful error messages.
- Testing approach: add an objective smoke test for every new endpoint and run offline
  validation tests when a live database or model is unnecessary.
- Testing approach: preserve reproducibility by using SID4=4098 and VERIFY_SEED=264098
  for generated experiments.
- Prohibited action: never commit passwords, API keys, session secrets, virtual
  environments, or generated cache directories.
- Prohibited action: do not delete or rewrite existing HW1-HW5 behavior when adding
  HW6 features.
- Memory rule: treat user-specific memory as scoped to the user_id and never mix
  records between users.
```

MEMORY.original.md - duplicate and contradiction excerpt

```
- Route 22 has frequent afternoon delays
- Route 22 has frequent afternoon delays
- Route 22 has frequent afternoon delays
- Main incident categories include delay and collision
- Main incident categories include delay and collision
- Reliable transit analysis is an important project goal
- Reliable transit analysis is an important project goal
- Response preference: concise
- Response preference: detailed
- Preferred model: qwen3:8b
- Preferred model: qwen3:4b
- Project status: active
```

```
  - Project status: paused
```

MEMORY.consolidated.md - excerpt (37 counted lines in total)

```
  # Consolidated Memory
  - Route 22 has frequent afternoon delays
  - Main incident categories include delay and collision
  - Reliable transit analysis is an important project goal
  - Conflict resolved for response preference: retained latest; superseded:
    - Response preference: concise
  - Response preference: detailed
  - Conflict resolved for preferred model: retained latest; superseded:
    - Preferred model: qwen3:8b
  - Preferred model: qwen3:4b
  - Conflict resolved for project status: retained latest; superseded:
    - Project status: active
  - Project status: paused
```

Each Conflict resolved line is a single line in the file. It is wrapped here only to fit the page.

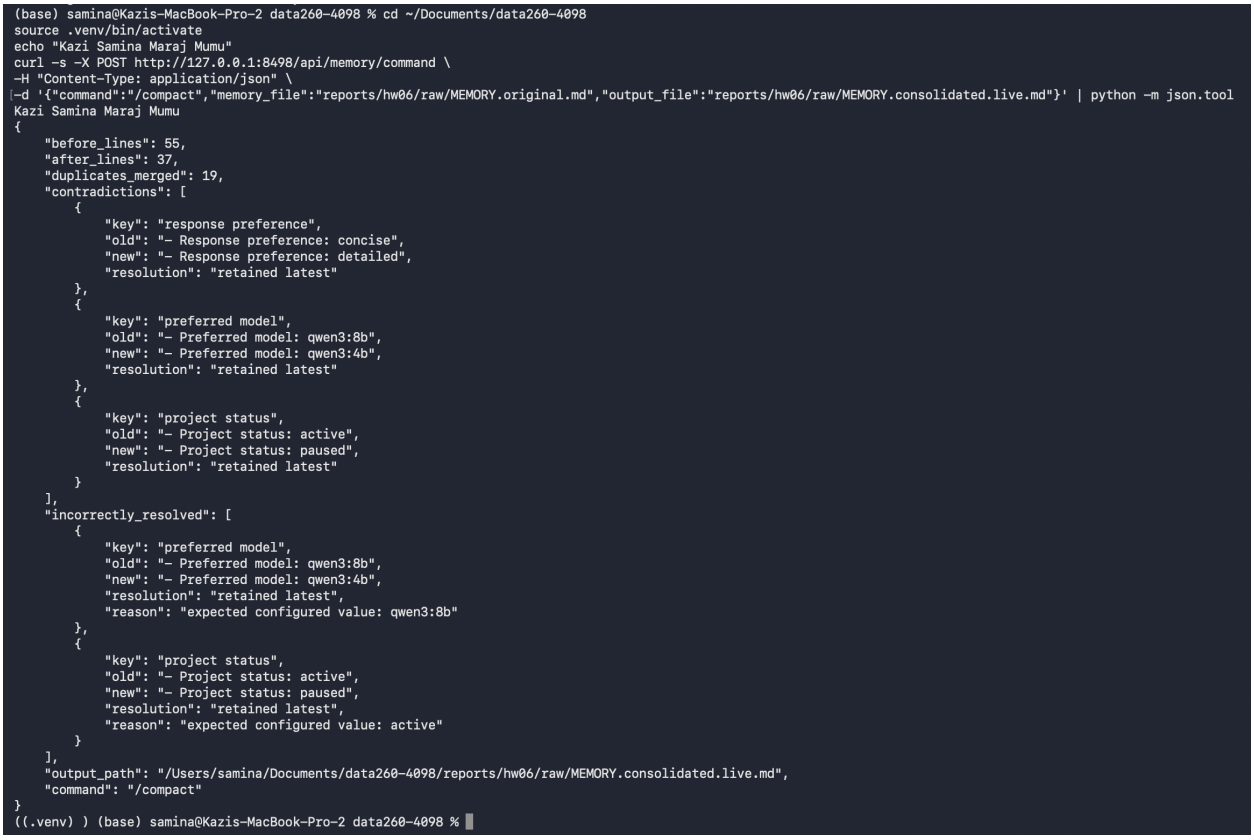


Figure 16. /compact run through the API on the 55-line fixture: 55 to 37 lines, 19 duplicates merged, 3 contradictions, 2 incorrectly resolved.



Figure 17. Earlier raw result before the manual contradiction audit (incorrectly_resolved was still []). The final line shows the AGENT.md instruction check passing for the generated function.

File Memory versus MongoDB Memory

Concern	MongoDB memory	File-based memory
Inspectability	Inspectable through mongosh, Compass, and the API endpoints; messages, summaries, and episodes are separate collections.	Very direct: AGENT.md and MEMORY.md can be opened, reviewed, and versioned as text.
Persistence after restart	Data survives API restarts in the MongoDB volume.	Data survives service and database restarts because it is stored in repository files.
Scalability	Better for many users and sessions, with queries, indexes, and concurrent writes.	Simple for one project, but manual consolidation becomes harder as files grow.
Facts that must not be forgotten	Stored as episodes or durable summaries and retrieved by user scope, but retrieval can miss them.	Stable project rules, safety rules, configuration, and critical preferences stay in durable files that are always loaded into the prompt.

MongoDB is better for conversation history and retrieval at scale, while files are easier to inspect and are better for stable instructions and facts that must never be dropped. The two approaches complement each other.

AI Use

1. I used an AI assistant to help plan the HW6 structure, draft MongoDB schemas and routes, and explain error messages. I ran, tested, and verified the code myself.

2. One result I independently checked was the memory embedding implementation. Python's built-in hash() is not stable between processes, so it was unsuitable for reproducible experiments.

3. I detected the issue by checking whether the same fact produced the same vector in a new Python process.

4. I changed the embedding index to use SHA-256, which produces deterministic vectors and makes the retrieval experiment reproducible.

Verification

make verify-hw06 records the homework number, SID4, the configuration values, the commit hash, and the model, and checks FastAPI health, the HW6 routes, a successful POST /api/chat reply, MongoDB Pydantic validation, and the required HW6 files. All checks passed.

```
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % git rev-parse HEAD
[git rev-parse hw6^{commit}
d756ec5362f0050acc0895719109afb9d57261ff
d756ec5362f0050acc0895719109afb9d57261ff
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % echo "Kazi Samina Maraj Mumu"
[make verify-hw06
Kazi Samina Maraj Mumu
Kazi Samina Maraj Mumu
{
  "homework": 6,
  "SID4": 4098,
  "PORT_BASE": 8498,
  "PREFIX": "s4098",
  "SEED": 4098,
  "VERIFY_SEED": 264098,
  "DOMAIN_ID": 2,
  "commit_hash": "d756ec5362f0050acc0895719109afb9d57261ff",
  "model": "qwen3:8b",
  "checks": [
    {
      "name": "FastAPI health",
      "passed": true
    },
    {
      "name": "HW6 routes registered",
      "passed": true
    },
    {
      "name": "POST /api/chat returns a reply",
      "passed": true
    },
    {
      "name": "Mongo Pydantic validation rejects missing fields",
      "passed": true
    },
    {
      "name": "Required HW6 files exist",
      "passed": true
    }
  ],
  "passed": true
}
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %
```

Figure 18. Final verification on commit d756ec5 (the hw6 tag): HEAD and hw6 match, and all five checks passed.

Conclusion

This homework added MongoDB CRUD functionality with validation, persistent conversational memory with summaries and episodic retrieval, file-based memory with consolidation commands, and memory-size experiments. The results show the tradeoff between prompt size, latency, cost, and answer quality.